



NAME: SAMARTH CHAND THAKURI

SAP ID: 500119430

BATCH: B1 CCVT

EXPERIMENT – 7

TITLE: Abstract Classes and Interfaces

1. In a 3D modeling application, define an abstract class named Solid that declares an abstract method calculateVolume(). Then, derive two concrete classes—Cuboid and Sphere—from the Solid class. Each subclass should override calculateVolume() to

compute its volume using the appropriate geometric formula. Finally, in your main method, create objects of Cuboid and Sphere and call their calculateVolume() methods to display the computed volumes.

Code:

```
import java.lang.Math;
abstract class Solid{
    abstract double calculateVolume();
}
class Cuboid extends Solid{
    private double lenght;
    private double breadth;
    private double height;
    public Cuboid(double lenght, double breadth, double height){
        this.breadth=breadth;
        this.lenght=lenght;
        this.height=height;
    }
    double calculateVolume(){
        return lenght*breadth*height;
    }
}
class Sphere extends Solid{
    private int radius;
    public Sphere(int radius){
        this.radius=radius;
    }
    double calculateVolume(){
        return 1.3*3.14*Math.pow(radius,3);
    }
}
```

```

}

public class First {
    public static void main(String a[]){
        Sphere sph=new Sphere(8);
        sph.calculateVolume();
    }
}

```

Consider an automotive application where you need to manage different types of vehicles. Create an interface called Vehicle with methods start(), stop(), and getFuelLevel(). Implement this interface in two classes, Car and Motorcycle, each providing its own implementation of the methods. Then write a main method that uses a Vehicle reference to hold objects of both Car and Motorcycle and calls the methods to demonstrate runtime polymorphism.

```

interface Vehicle{
    void start();
    void stop();
    int getFuelLevel();
}

class Car implements Vehicle{
    public void start(){
        System.out.println("Car starts");
    }
    public void stop(){
        System.out.println("Stop the car");
    }
    public int getFuelLevel(){

```

```
        return 100;
    }
}

class Motorcycle implements Vehicle{
    public void start(){
        System.out.println("MotorCycle starts");
    }
    public void stop(){
        System.out.println("Stop the MotorCycle");
    }
    public int getFuelLevel(){
        return 100;
    }
}

public class Second {
    public static void main(String args[]){

    }
}
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab-> & 'C:\Program Files\Java\jdk-17\bin\java.exe' -jar 'C:\Users\samar\AppData\Roaming\Code\User\workspaceStorage\abb8d221f0bb4c8398f033aa5831e6bc\redhat.java\jdt_ws\OOPS-lab-lab_16c6cd2b\bin' 'Second'
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab-> ^C
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab->
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab-> c:;; cd 'c:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab-lab_16c6cd2b\bin' 'Second'
PS-lab-_16c6cd2b\bin' 'Second'
Car starts
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab->
```

Design an interface called Wallet that declares two methods: addFunds(double amount) and spendFunds(double amount). Implement this interface in a class named DigitalWallet that maintains a private balance variable. Ensure that the class provides controlled access to update the balance only through the interface methods. Then, write a separate class with a main method that creates a DigitalWallet object, performs a series of fund additions and expenditures, and prints the updated balance to verify that external classes cannot directly manipulate the wallet's internal data.

```
interface Wallet {
    void addFunds(double amount);
```

```
        void spendFunds(double amount) ;
    }

    class DigitalWallet implements Wallet {
        private double balance;

        public DigitalWallet() {
            this.balance = 0.0;
        }

        public void addFunds(double amount) {
            if (amount > 0) {
                balance += amount;
                System.out.println("Added Rs." + amount + " to the
wallet.");
            } else {
                System.out.println("Invalid deposit amount.");
            }
        }

        public void spendFunds(double amount) {
            if (amount > 0 && amount <= balance) {
                balance -= amount;
                System.out.println("Spent $" + amount + " from the
wallet.");
            } else if (amount > balance) {
                System.out.println("Insufficient funds.");
            } else {
                System.out.println("Invalid spending amount.");
            }
        }

        public void displayBalance() {
            System.out.println("Current balance: Rs." + balance);
        }
    }
}
```

```
    }  
}  
  
public class Third {  
    public static void main(String[] args) {  
        DigitalWallet myWallet = new DigitalWallet();  
  
        myWallet.addFunds(100.0);  
        myWallet.displayBalance();  
  
        myWallet.spendFunds(30.0);  
        myWallet.displayBalance();  
  
        myWallet.spendFunds(80.0);  
        myWallet.displayBalance();  
  
        myWallet.addFunds(50.0);  
        myWallet.displayBalance();  
    }  
}
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab-> & 'C:\Program Files\Java\jdk-
ing\Code\User\workspaceStorage\abb8d221f0bb4c8398f033aa5831e6bc\redhat.java\jdt_ws\OOPS-
Added Rs.100.0 to the wallet.
Current balance: Rs.100.0
Spent $30.0 from the wallet.
Current balance: Rs.70.0
Insufficient funds.
Current balance: Rs.70.0
Added Rs.50.0 to the wallet.
Current balance: Rs.120.0
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab->
```

Create an interface named Remote with methods powerOn(), powerOff(), and changeChannel(int channel). Next, implement this interface in a class called Television, where each method prints a message indicating its action (e.g., "TV is now on", "TV is off", "Channel changed to X"). Finally, develop a class with a main method that instantiates a Television object and uses a Remote reference to call the methods, thereby demonstrating how polymorphism and interface abstraction allow you to control a device through a common interface.

```
interface Remote {
    void powerOn();
    void powerOff();
    void changeChannel(int channel);
}

class Television implements Remote {
    public void powerOn() {
        System.out.println("TV is now on");
    }

    public void powerOff() {
        System.out.println("TV is off");
    }

    public void changeChannel(int channel) {
        System.out.println("Channel changed to " + channel);
    }
}

public class Fourth {
    public static void main(String[] args) {
        Television tv = new Television();
        Remote remote = tv;

        remote.powerOn();
        remote.changeChannel(5);
        remote.changeChannel(10);
        remote.powerOff();
    }
}
```

PROBLEMS 1

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab-> & 'C:\Program Files\Java\jdk-17\bin\java.  
ing\Code\User\workspaceStorage\abb8d221f0bb4c8398f033aa5831e6bc\redhat.java\jdt_ws\OOPS-lab-_16c6cd2  
TV is now on  
Channel changed to 5  
Channel changed to 10  
TV is off  
PS C:\Users\samar\OneDrive\ドキュメント\GitHub\OOPS-lab->
```